EF-101 - Fall 2026

System Understanding Project Handbook

Software, Hardware, Data, Networking and Information Systems

Project Team: Exactly 3 students per group
Submission Week: Week 13
Submission Date: Monday, 9 November 2026

Purpose of the project

The purpose of these projects is not to make students expert programmers or specialists in a particular API, framework or development platform. The purpose is to help students understand how practical computing systems work as complete systems: user interface, software logic, hardware, storage, databases, networking, web services, data flow, testing and decision making.

Course project handbook prepared for classroom use.

1. General Project Philosophy

Each group will be assigned one project from this handbook. Students are free to select reasonable technologies, programming languages, libraries, development environments and hosting services unless a project explicitly specifies otherwise. The emphasis is on **functional understanding of the complete system**.

> **Central rule:** A student may use ChatGPT, other generative AI systems, GitHub examples, online tutorials, libraries, APIs, templates and development tools. However, every group member must understand the submitted project at component level and must be able to explain what each major component does, how data moves through the system, where data is stored, what external services are used, and what happens when the user performs an important action.

Students are not expected to memorize every line of code. They are expected to understand the architecture, logic and behavior of the system they submit.

2. Common Requirements for All Projects

2.1 Group Composition

- Each project is to be completed by a **group of exactly three students**, unless the instructor gives written permission for a different arrangement.
- All three members must contribute meaningfully.
- The group must maintain a short contribution record showing which member worked on which parts.
- During evaluation, questions may be directed to any individual member. One member cannot act as the sole spokesperson for the whole group.

2.2 Submission Deadline

- Projects are due in **Week 13**.
- The stated submission date is **Monday, 9 November 2026**.
- The working system, source/project files, report and supporting evidence must be available by the deadline.
- Demonstration or viva scheduling may be distributed across Week 13 by the instructor, but the project itself must be submission-ready on 9 November 2026.

2.3 Minimum Deliverables

Every group must submit all items below unless a project specifically states that an item is not applicable.

1. **Working project**: a functional prototype, application, experimental setup, dashboard or analysis that can be demonstrated.
2. **Source/project files**: all code, configuration files, notebooks, spreadsheets, database files, design files and other files necessary to reproduce the work.
3. **Project report**: recommended length 6–10 pages excluding appendices. It must explain the problem, objectives, architecture, technologies, implementation, testing, results, limitations and individual contributions.
4. **System architecture diagram**: a clear diagram showing major components and arrows indicating data or control flow.

5. **Data/storage description**: explain where information is stored, in what form, and why that storage method was selected.
6. **Testing evidence**: screenshots, logs, tables, test cases or measurement results showing that the project was tested.
7. **Demonstration dataset**: enough sample records, users, products, teams, logs, measurements or transactions to demonstrate all important features.
8. **README / run instructions**: concise steps explaining how the instructor can start or inspect the project.
9. **Tool and AI disclosure**: list major libraries, APIs, frameworks, online services, code sources and AI tools used. This is not a penalty; undisclosed copying is a problem.
10. **Contribution statement**: one short table showing the meaningful contribution of each of the three members.

2.4 Mandatory System Understanding

Every group must be prepared to explain, where applicable:

- What happens from the moment the user presses a button until the output appears.
- Which parts run on the user's device and which parts run elsewhere.
- Whether the project uses a client, server, database, local file, cloud service, API, sensor, microcontroller or network.
- What data enters the system, how it is processed, where it is stored and how it is retrieved.
- What would stop working if the Internet were disconnected.
- What would happen if the database or storage file were deleted.
- Why the selected platform, language, library or service was used.
- The meaning of key terms used in the project, such as IP address, port, database table, API request, JSON, local storage, CPU utilization, RAM, sensor, client, server or authentication.
- At least one important limitation of the submitted implementation.

2.5 Live Demonstration Expectations

During evaluation, the instructor may ask the group to:

- add a new record and show where it appears;
- change a configuration value, label, threshold or rule;
- intentionally enter incorrect data and explain how the project handles it;
- trace one complete operation through the architecture;
- identify the relevant database table, file, function, API call or hardware component;
- make one small modification to prove that the group understands the project rather than merely running a copied solution.

2.6 Quality, Ethics and Responsible Use

- Do not use unauthorized personal information or confidential data.
- Do not attack, scan, intercept or interfere with networks, devices or accounts for which you do not have permission.
- Do not use copyrighted game graphics, music or branding without permission. Original or openly licensed assets are preferred.
- Do not require paid APIs or paid hosting if a free/local alternative can satisfy the project requirements.
- If a project involves hardware stress testing, electricity or physical sensors, follow manufacturer limits and laboratory safety instructions.
- A visually attractive interface is useful, but appearance cannot substitute for correct functionality and understanding.

3. Recommended Evaluation Structure

The exact marking policy may be adjusted by the instructor. A recommended structure is shown below.

Criterion	Suggested Weight	What is being assessed
Functional completeness	25%	Mandatory requirements work correctly and can be demonstrated.
System architecture and component understanding	20%	Group understands software, hardware, storage, network and service components.
Data flow, storage and processing	15%	Correct handling of information, calculations, records, logs or measurements.
Testing and evidence	10%	Meaningful test cases, logs, measurements and handling of errors/edge cases.
Documentation and architecture diagram	10%	Clear report, reproducible instructions and accurate system diagram.
Usability and presentation	5%	Interface or experimental presentation is understandable and organized.
Individual viva and live modification	15%	Each member demonstrates personal understanding and can explain/modify the work.

4. Project Catalogue

ID	Project	Primary concepts

EF101-P01	Web-Based Hangman Game with Leaderboard	Web, hosting, database, frontend/backend, scoring
EF101-P02	Cricket Match Scoring Android App	Mobile app, local database, application state, rules engine
EF101-P03	Fleet Fuel and Expense Manager	Data management, calculations, dashboards, visualization
EF101-P04	Computer Hardware Stress Testing and Resource Analysis	CPU, RAM, GPU, storage, logging, performance
EF101-P05	Python-Based 2D Game	Programming, events, game loop, graphics, files
EF101-P06	URL Shortener with Analytics	Web, URLs, HTTP redirects, database, usage analytics
EF101-P07	Classroom/Laboratory Booking System	Web/database, scheduling, conflict detection, information systems
EF101-P08	Personal Expense and Budget Dashboard	Data entry, analytics, visualization, decision support
EF101-P09	Local Network Design and Performance Investigation	Router, switch, IP, DHCP, latency, bandwidth
EF101-P10	QR-Based Student Attendance System	QR, database, client/server, reporting
EF101-P11	Computer Hardware Inventory and Maintenance System	Hardware literacy, database, asset management
EF101-P12	Weather API Data Collection and Visualization System	API, JSON, Internet services, database, visualization
EF101-P13	Network Traffic Observation and Analysis	Packets, DNS, TCP, TLS, Wireshark, networking
EF101-P14	Online Restaurant Ordering System	Web/mobile, database, cart, workflow, user roles
EF101-P15	Digital Library Management System	Database, search, transactions, issue/return workflow
EF101-P16	PC Build and Performance Comparison	Hardware compatibility, cost, performance, decision making
EF101-P17	File Storage, Compression and Backup Investigation	Storage, file formats, compression, integrity, backup
EF101-P18	Mini E-Commerce Store	Web, product database, cart, inventory, orders
EF101-P19	Digital Photo and File Organizer	File systems, metadata, hashing, storage analytics, local application
EF101-P20	Mini Client-Server Data Communication System	Client/server, IP, port, network, storage

EF101-P17: File Storage, Compression and Backup Investigation

Project ID: EF101-P17 **Group size:** 3 students
Submission: Week 13, Monday, 9 November 2026

Objective

Experimentally investigate how file type affects storage, compression, transfer and backup. The project should connect abstract terms such as bits, bytes, formats and compression to measurements performed by the students.

Mandatory Dataset

Create or collect at least 12 test files covering multiple types, including plain text/document, bitmap or uncompressed image, JPEG/PNG, uncompressed audio if available, compressed audio, video and already-compressed/archive data.

Mandatory Experiments

- Record original size of every test file in bytes/KB/MB as appropriate.
- Compress files individually and/or in groups using a standard lossless archive tool such as ZIP/7-Zip.
- Calculate compression ratio or percentage size reduction.
- Compare why text/BMP may compress differently from JPEG/MP3/video.
- Copy a sufficiently large test set between two storage locations and measure transfer time and approximate throughput.
- Where possible compare two storage media or locations, such as SSD vs USB drive or local vs cloud upload.
- Demonstrate data integrity using file hashes/checksums before and after a copy for at least two files.
- Design a backup plan for a hypothetical student's important data using local, external and/or cloud storage.
- Explain the difference between backup and synchronization.

Required Analysis

Use tables and graphs to compare original size, compressed size, compression ratio and transfer performance. Explain lossless vs lossy compression and why repeatedly compressing an already compressed format may provide little benefit.

Backup Requirement

The final report must include a simple 3-2-1 backup interpretation: three copies of important data, on two different media/types, with one copy kept separately/off-site or in a suitable cloud location.

Bonus / Extension: Compare multiple archive algorithms, encrypt a backup archive and discuss key management, compare cloud upload/download performance, or automate checksum generation using a script.